

BÀI THỰC HÀNH: DANH SÁCH LIÊN KẾT ĐƠN VÀ KÉP

1. MỤC TIÊU BÀI HỌC

- Hiểu cấu trúc bộ nhớ và nguyên lý hoạt động của danh sách liên kết đơn (Single Linked List) và danh sách liên kết kép (Doubly Linked List).
- Cài đặt được các phương thức cơ bản: thêm, xóa, duyệt, tìm kiếm, chèn vào vị trí bất kỳ.
- So sánh ưu, nhược điểm giữa hai loại danh sách.
- Vận dụng để giải quyết các bài toán ứng dụng như: quản lý danh sách sinh viên, hàng đợi, hoặc danh sách lùi – tiến trong trình duyệt.

2. LÝ THUYẾT NGẮN GỌN

Single Linked List

Là cấu trúc dữ liệu gồm các node, mỗi node gồm: data và next (con trỏ trỏ tới node kế tiếp).

Đặc điểm: chỉ có thể duyệt theo một chiều (từ đầu đến cuối). Thêm/xóa đầu nhanh, thêm/xóa cuối chậm.

Doubly Linked List

Là danh sách liên kết có mỗi node gồm: data, prev (trỏ tới node trước), next (trỏ tới node sau).

Đặc điểm: Duyệt được hai chiều, tốn nhiều bộ nhớ hơn nhưng hỗ trợ thao tác xóa/chèn linh hoạt hơn.

3. BÀI THỰC HÀNH

Phần A – Cấu trúc dữ liệu

Node cho Single Linked List:

```
class Node {  
    int data;  
    Node next;  
    Node(int data) {  
        this.data = data;  
        this.next = null;  
    }  
}
```

Node cho Doubly Linked List:

```
class DNode {  
    int data;  
    DNode prev;  
    DNode next;  
    DNode(int data) {  
        this.data = data;  
        this.prev = null;  
    }  
}
```

```
        this.next = null;
    }
}
```

Phần B – Cài đặt Single Linked List (Java)

```
class Node {
    int data;
    Node next;
    Node(int data) {
        this.data = data;
        this.next = null;
    }
}
```

```
class SingleLinkedList {
    Node head;

    // 1. Thêm đầu
    public void addFirst(int data) {
        Node newNode = new Node(data);
        newNode.next = head;
        head = newNode;
    }

    // 2. Thêm cuối
    public void addLast(int data) {
        Node newNode = new Node(data);
        if (head == null) {
            head = newNode;
            return;
        }
        Node current = head;
        while (current.next != null) {
            current = current.next;
        }
        current.next = newNode;
    }

    // 3. Xóa đầu
    public void removeFirst() {
        if (head != null)
            head = head.next;
    }

    // 4. Xóa cuối
    public void removeLast() {
        if (head == null || head.next == null) {
            head = null;
            return;
        }
    }
}
```

```

    }
    Node current = head;
    while (current.next.next != null) {
        current = current.next;
    }
    current.next = null;
}

// 5. Duyệt danh sách
public void traverse() {
    Node current = head;
    while (current != null) {
        System.out.print(current.data + " → ");
        current = current.next;
    }
    System.out.println("null");
}
}

public class TestSingleList {
    public static void main(String[] args) {
        SingleLinkedList list = new SingleLinkedList();
        list.addFirst(10);
        list.addLast(20);
        list.addLast(30);
        list.addFirst(5);
        System.out.print("Danh sách: ");
        list.traverse();

        list.removeFirst();
        list.removeLast();
        System.out.print("Sau khi xóa: ");
        list.traverse();
    }
}

```

KẾT QUẢ CHẠY KIỂM THỬ (SingleLinkedList):

Danh sách: 5 → 10 → 20 → 30 → null

Sau khi xóa: 10 → 20 → null

Phần C – Cài đặt Doubly Linked List (Java)

```

class DNode {
    int data;
    DNode prev;
    DNode next;
    DNode(int data) {
        this.data = data;
        this.prev = null;
        this.next = null;
    }
}

```

```
}  
}
```

```
class DoublyLinkedList {
```

```
    DNode head, tail;
```

```
    // 1. Thêm đầu
```

```
    public void addFirst(int data) {
```

```
        DNode newNode = new DNode(data);
```

```
        if (head == null) {
```

```
            head = tail = newNode;
```

```
        } else {
```

```
            newNode.next = head;
```

```
            head.prev = newNode;
```

```
            head = newNode;
```

```
        }
```

```
    }
```

```
    // 2. Thêm cuối
```

```
    public void addLast(int data) {
```

```
        DNode newNode = new DNode(data);
```

```
        if (tail == null) {
```

```
            head = tail = newNode;
```

```
        } else {
```

```
            tail.next = newNode;
```

```
            newNode.prev = tail;
```

```
            tail = newNode;
```

```
        }
```

```
    }
```

```
    // 3. Xóa đầu
```

```
    public void removeFirst() {
```

```
        if (head == null) return;
```

```
        head = head.next;
```

```
        if (head != null) head.prev = null;
```

```
        else tail = null;
```

```
    }
```

```
    // 4. Xóa cuối
```

```
    public void removeLast() {
```

```
        if (tail == null) return;
```

```
        tail = tail.prev;
```

```
        if (tail != null) tail.next = null;
```

```
        else head = null;
```

```
    }
```

```
    // 5. Duyệt xuôi
```

```
    public void traverseForward() {
```

```
        DNode current = head;
```

```
        while (current != null) {
```

```
            System.out.print(current.data + " ↔ ");
```

```

        current = current.next;
    }
    System.out.println("null");
}

// 6. Duyệt ngược
public void traverseBackward() {
    DNode current = tail;
    while (current != null) {
        System.out.print(current.data + " ↔ ");
        current = current.prev;
    }
    System.out.println("null");
}
}

public class TestDoublyList {
    public static void main(String[] args) {
        DoublyLinkedList list = new DoublyLinkedList();
        list.addFirst(10);
        list.addLast(20);
        list.addLast(30);
        list.addFirst(5);

        System.out.print("Duyệt xuôi: ");
        list.traverseForward();

        System.out.print("Duyệt ngược: ");
        list.traverseBackward();

        list.removeFirst();
        list.removeLast();
        System.out.print("Sau khi xóa xuôi: ");
        list.traverseForward();
    }
}

```

KẾT QUẢ CHẠY KIỂM THỬ (DoublyLinkedList):

Duyệt xuôi: 5 ↔ 10 ↔ 20 ↔ 30 ↔ null
 Duyệt ngược: 30 ↔ 20 ↔ 10 ↔ 5 ↔ null
 Sau khi xóa xuôi: 10 ↔ 20 ↔ null

4. BÀI TẬP ỨNG DỤNG

- Viết phương thức `insertAtPosition(int data, int position)` để chèn một node vào vị trí bất kỳ trong danh sách đơn.
- Viết phương thức `find(int value)` để tìm kiếm một phần tử trong danh sách liên kết kép.
- Cài đặt chương trình quản lý danh sách sinh viên sử dụng Doubly Linked List, cho phép thêm, xóa và hiển thị danh sách sinh viên từ đầu đến cuối và ngược lại.
- Mô phỏng chức năng 'Back' và 'Forward' trong trình duyệt web bằng Doubly Linked List.